

Nota: Este material complementar, disponível em <https://prettore.github.io/lectures.html> representa uma cópia resumida de conteúdos bibliográficos disponíveis gratuitamente na Internet.

Expressões Aritméticas, Lógicas e Relacionais

Introdução.....	1
Expressões Aritméticas.....	1
Expressões Relacionais.....	2
Expressões Lógicas.....	3
Combinação de Expressões.....	4
Precedência entre Diferentes Tipos de Operadores.....	4
Expressões em Diferentes Linguagens de Programação.....	4
Boas Práticas no Uso de Expressões.....	5
Conclusão.....	5
Referências.....	5

Introdução

As expressões aritméticas, lógicas e relacionais são elementos fundamentais na programação de computadores, permitindo a manipulação de dados e a tomada de decisões em algoritmos. Estas expressões são construídas a partir de operadores que relacionam valores para produzir novos resultados. Compreender como funcionam esses operadores e como construir expressões válidas é essencial para o desenvolvimento de programas eficientes e corretos. Este material explora os principais tipos de expressões utilizadas em programação, seus operadores, regras de precedência e aplicações práticas.

Expressões Aritméticas

As expressões aritméticas são utilizadas para realizar cálculos matemáticos em programas de computador. Elas são compostas por operandos (valores numéricos ou variáveis) e operadores aritméticos que definem as operações a serem realizadas.

Os principais operadores aritméticos são:

- Adição (+): Soma dois valores.
Exemplo: $5 + 3 = 8$
- Subtração (-): Subtrai o segundo valor do primeiro.
Exemplo: $10 - 4 = 6$
- Multiplicação (*): Multiplica dois valores.
Exemplo: $3 * 4 = 12$
- Divisão (/): Divide o primeiro valor pelo segundo.
Exemplo: $20 / 5 = 4$
- Módulo (%): Retorna o resto da divisão inteira do primeiro valor pelo segundo.
Exemplo: $14 \% 3 = 2$ (pois 14 dividido por 3 é 4 com resto 2)
- Potenciação (** ou ^): Eleva o primeiro valor à potência do segundo.
Exemplo: $2 ^ 3 = 8$ (2 elevado à terceira potência)

Além destes, existem operadores aritméticos unários, que operam sobre um único valor:

- Negação (-): Inverte o sinal de um número.
Exemplo: -5 (negativo de 5)
- Incremento (++): Aumenta o valor de uma variável em 1.
Exemplo: x++ (equivale a $x = x + 1$)
- Decremento (--): Diminui o valor de uma variável em 1.
Exemplo: x-- (equivale a $x = x - 1$)

Na avaliação de expressões aritméticas complexas, é importante considerar a precedência dos operadores. A ordem padrão de avaliação é:

- Parênteses (expressões entre parênteses são avaliadas primeiro)
- Potenciação
- Multiplicação, divisão e módulo (da esquerda para a direita)
- Adição e subtração (da esquerda para a direita)

Por exemplo, na expressão $2 + 3 * 4$, a multiplicação tem precedência sobre a adição, então o resultado é 14 (e não 20, que seria o resultado se a adição fosse realizada primeiro).

Para alterar a ordem de avaliação, podem ser utilizados parênteses. Na expressão $(2 + 3) * 4$, os parênteses indicam que a adição deve ser realizada primeiro, resultando em 20.

Expressões Relacionais

As expressões relacionais são utilizadas para comparar valores e determinar a relação entre eles. O resultado de uma expressão relacional é sempre um valor lógico (verdadeiro ou falso).

Os principais operadores relacionais são:

- Igual a (==): Verifica se dois valores são iguais.
Exemplo: $5 == 5$ (verdadeiro)
- Diferente de (!= ou <>): Verifica se dois valores são diferentes.
Exemplo: $5 != 3$ (verdadeiro)
- Maior que (>): Verifica se o primeiro valor é maior que o segundo.
Exemplo: $7 > 4$ (verdadeiro)
- Menor que (<): Verifica se o primeiro valor é menor que o segundo.
Exemplo: $2 < 6$ (verdadeiro)
- Maior ou igual a (>=): Verifica se o primeiro valor é maior ou igual ao segundo.
Exemplo: $8 >= 8$ (verdadeiro)
- Menor ou igual a (<=): Verifica se o primeiro valor é menor ou igual ao segundo.
Exemplo: $3 <= 5$ (verdadeiro)

As expressões relacionais são frequentemente utilizadas em estruturas de controle condicional, como if-else, para determinar o fluxo de execução de um programa.

É importante notar que, em muitas linguagens de programação, o operador de igualdade (==) é diferente do operador de atribuição (=). O primeiro compara valores, enquanto o segundo atribui um valor a uma variável.

Expressões Lógicas

As expressões lógicas são utilizadas para combinar ou modificar valores lógicos (verdadeiro ou falso). Elas são fundamentais para a implementação de lógica condicional em programas.

Os principais operadores lógicos são:

- E lógico (AND, &, E): Retorna verdadeiro apenas se ambos os operandos forem verdadeiros.

Tabela-verdade:

- Verdadeiro AND Verdadeiro = Verdadeiro
- Verdadeiro AND Falso = Falso
- Falso AND Verdadeiro = Falso
- Falso AND Falso = Falso

- OU lógico (OR, ||, OU): Retorna verdadeiro se pelo menos um dos operandos for verdadeiro.

Tabela-verdade:

- Verdadeiro OR Verdadeiro = Verdadeiro
- Verdadeiro OR Falso = Verdadeiro
- Falso OR Verdadeiro = Verdadeiro
- Falso OR Falso = Falso

- NÃO lógico (NOT, !, NÃO): Inverte o valor lógico do operando.

Tabela-verdade:

- NOT Verdadeiro = Falso
- NOT Falso = Verdadeiro

Além destes, existem operadores lógicos menos comuns, mas que podem ser encontrados em algumas linguagens ou contextos específicos:

- OU exclusivo (XOR, ^): Retorna verdadeiro se exatamente um dos operandos for verdadeiro.

Tabela-verdade:

- Verdadeiro XOR Verdadeiro = Falso
- Verdadeiro XOR Falso = Verdadeiro
- Falso XOR Verdadeiro = Verdadeiro
- Falso XOR Falso = Falso

- NÃO-E (NAND): Retorna falso apenas se ambos os operandos forem verdadeiros.

Tabela-verdade:

- Verdadeiro NAND Verdadeiro = Falso
- Verdadeiro NAND Falso = Verdadeiro
- Falso NAND Verdadeiro = Verdadeiro
- Falso NAND Falso = Verdadeiro

- NÃO-OU (NOR): Retorna verdadeiro apenas se ambos os operandos forem falsos.

Tabela-verdade:

- Verdadeiro NOR Verdadeiro = Falso
- Verdadeiro NOR Falso = Falso
- Falso NOR Verdadeiro = Falso
- Falso NOR Falso = Verdadeiro

Na avaliação de expressões lógicas complexas, também é importante considerar a precedência dos operadores. **A ordem padrão de avaliação é:** 1. NOT (maior precedência), 2. AND, 3. OR (menor precedência)

Por exemplo, na expressão `A OR B AND C`, o operador `AND` tem precedência sobre o operador `OR`, então a expressão é avaliada como `A OR (B AND C)`.

Combinação de Expressões

Em programação, é comum combinar diferentes tipos de expressões para criar condições mais complexas. Por exemplo, podemos combinar expressões aritméticas e relacionais para verificar se o resultado de um cálculo atende a determinada condição:

```
...
(x + y) > 10
...
```

Esta expressão primeiro calcula a soma de `x` e `y` (expressão aritmética) e depois verifica se o resultado é maior que 10 (expressão relacional).

Também podemos combinar expressões relacionais e lógicas para criar condições compostas:

```
...
(idade >= 18) AND (temCarteira == verdadeiro)
...
```

Esta expressão verifica se a idade é maior ou igual a 18 E se a pessoa tem carteira de motorista.

Ao combinar diferentes tipos de expressões, é importante considerar a precedência dos operadores e utilizar parênteses quando necessário para garantir que as operações sejam realizadas na ordem desejada.

Precedência entre Diferentes Tipos de Operadores

Quando uma expressão contém diferentes tipos de operadores, a ordem de avaliação segue uma hierarquia de precedência:

1. Operadores aritméticos (maior precedência)
2. Operadores relacionais
3. Operadores lógicos (menor precedência)

Por exemplo, na expressão `A + B > C AND D == E`, a ordem de avaliação seria:

1. Calcular `A + B` (operador aritmético)
2. Verificar se `(A + B) > C` (operador relacional)
3. Verificar se `D == E` (operador relacional)
4. Combinar os resultados das comparações com `AND` (operador lógico)

Expressões em Diferentes Linguagens de Programação

Embora os conceitos fundamentais de expressões aritméticas, lógicas e relacionais sejam semelhantes em todas as linguagens de programação, a sintaxe específica pode variar. Alguns exemplos:

C/C++	Python	Java	JavaScript
<code>int resultado = (a + b) * c; if (x > 0 && y < 10) {</code>	<code>resultado = (a + b) * c if x > 0 and y < 10:</code>	<code>int resultado = (a + b) * c; if (x > 0 && y < 10) {</code>	<code>let resultado = (a + b) * c; if (x > 0 && y < 10) {</code>

// código a ser executado }	# código a ser executado	// código a ser executado }	// código a ser executado }
--------------------------------	--------------------------	--------------------------------	--------------------------------

Boas Práticas no Uso de Expressões

Para garantir a clareza e a correção de expressões em programas, algumas boas práticas devem ser seguidas:

- Utilize parênteses para tornar explícita a ordem de avaliação, mesmo quando não são estritamente necessários.
Exemplo: $(a + b) * c$ em vez de $a + b * c$
- Evite expressões muito complexas. Se uma expressão se tornar difícil de entender, divida-a em partes menores e utilize variáveis intermediárias.
- Tenha cuidado com comparações de igualdade em valores de ponto flutuante, pois erros de arredondamento podem levar a resultados inesperados.
- Esteja atento a possíveis divisões por zero em expressões aritméticas.
- Verifique se os tipos de dados dos operandos são compatíveis com os operadores utilizados.
- Utilize nomes de variáveis significativos para tornar as expressões mais legíveis.
- Documente expressões complexas com comentários explicativos.

Conclusão

As expressões aritméticas, lógicas e relacionais são componentes fundamentais da programação, permitindo a manipulação de dados e a implementação de lógica condicional em algoritmos. Compreender como funcionam os diferentes tipos de operadores, suas regras de precedência e como combiná-los para criar expressões complexas é essencial para o desenvolvimento de programas eficientes e corretos.

Neste material, exploramos os principais tipos de expressões utilizadas em programação, seus operadores e regras de avaliação. Também discutimos como combinar diferentes tipos de expressões e apresentamos boas práticas para garantir a clareza e a correção de expressões em programas.

O domínio desses conceitos é fundamental para avançar no estudo de estruturas de controle, como condicionais e loops, que utilizam expressões para determinar o fluxo de execução de um programa.

Referências

-  Livros e Apostilas
 - CORMEN, T. H. Introduction to Algorithms. MIT Press.
 - GOODRICH, M. Data Structures and Algorithms in Python.
 - Tenenbaum, A. M. Estruturas de Dados e Algoritmos em C
 - P. Feofiloff. Algoritmos em Linguagem C. Campus-Elsevier, 1a. edição, 2009 H. M. Deitel, P. J. Deitel. C - Como Programar, 6a. edição, Pearson Education, 2011.

- B. W. Kernighan, D. M. Ritchie. The C Programming Language, 2a. edição, Prentice-Hall, 1988 [Tradução: C - A Linguagem de Programação. Editora Campus, 1989].
- J. L. Szwarcfter, L. Markenzon. Estruturas de Dados e seus Algoritmos, 3a. edição, Editora LTC, 2010.
- W. Celes, R. Cerqueira, J.L. Rangel. Introdução a Estruturas de Dados, 1a. edição, Editora Campus, 2004.
- N. Ziviani. Projeto de Algoritmos com Implementações em Pascal e C, 3a. edição, Editora Cengage Learning, 2011.
- T. Cormen, C. Leiserson, R. Rivest, C. Stein. Algoritmos - Teoria e Prática, 3a. edição, Editora Campus, 2012.
- R. Sedgewick, K. Wayne. Algorithms, 4a. edição, Addison -Wesley, 2011.
- A. Kelley, I. Pohl. A Book on C, 4a. edição, Addison Wesley, 1998.
-  Recursos Online
 - Expressões aritméticas, relacionais e lógicas - SlideShare - <https://pt.slideshare.net/slideshow/expressoes-aritmticas-relacionais-e-lgicas-245557385/245557385>
 - Expressões aritméticas, relacionais e lógicas - IME-USP - <https://panda.ime.usp.br/cc110/static/cc110/02-expressoes.html>
 - Operações relacionais e lógicas - Embarcados - <https://embarcados.com.br/operacoes-relacionais-e-logicas/>
 - Operadores e Expressões - IBM - <https://www.ibm.com/docs/pt-br/tcamfma/6.3.0?topic=tesl-operators-expressions>
 - Operadores - Minicurso de Lógica de Programação - <https://mclp.dicasdeprogramacao.com.br/licao-4-operadores/>
 - GeeksforGeeks - Operators in C/C++
 - W3Schools - JavaScript Operators
-  Vídeos e Cursos
- Curso em Vídeo - Operadores Aritméticos, Relacionais e Lógicos
- Algoritmos e Programação de Computadores I - Expressões lógicas e operadores - <https://www.youtube.com/watch?v=qRXno770Mfs>
- Khan Academy - Introdução à Programação

Isenção de Responsabilidade:

Os autores deste documento não reivindicam a autoria do conteúdo original compilado das fontes mencionadas. Este documento foi elaborado para fins educativos e de referência, e todos os créditos foram devidamente atribuídos aos respectivos autores e fontes originais.

Qualquer utilização comercial ou distribuição do conteúdo aqui compilado deve ser feita com a devida autorização dos detentores dos direitos autorais originais. Os compiladores deste documento não assumem qualquer responsabilidade por eventuais violações de direitos autorais ou por quaisquer danos decorrentes do uso indevido das informações contidas neste documento.

Ao utilizar este documento, o usuário concorda em respeitar os direitos autorais dos autores originais e isenta os compiladores de qualquer responsabilidade relacionada ao conteúdo aqui apresentado.